Introducción a la Ingeniería de Datos

Maestría en Inteligencia Artificial - MIA 03

Facultad Politécnica - Universidad Nacional de Asunción

Clase 6: Calidad y Testing de Datos

Contenidos

¿Por Qué Importa la Calidad de Datos? 3
Tests Nativos de dbt 9
dbt-expectations 21
Singular Tests 36
Ejecutar Tests 45
Documentación en dbt 50
Actividad Asincrónica 57

¿Por Qué Importa la Calidad de Datos?

El Costo de los Datos Malos

Garbage In, Garbage Out

Los modelos de ML y dashboards son tan buenos como los **datos que los alimentan**.

Datos de mala calidad llevan a:

- Decisiones de negocio incorrectas
- Modelos de ML con predicciones erróneas
- Pérdida de confianza en los datos
- Retrabajo y costos adicionales

Dimensiones de la Calidad de Datos

Dimensión	Descripción
Completitud	¿Están todos los datos que deberían estar?
Unicidad	¿Hay duplicados donde no debería haberlos?
Validez	¿Los valores están en rangos/formatos válidos?
Consistencia	¿Los datos son coherentes entre tablas?
Precisión	¿Los datos reflejan la realidad correctamente?
Frescura	¿Los datos están actualizados?

Testing de Datos vs Testing de Software

Testing de Software

- Código determinístico
- Input conocido → Output esperado
- Tests pasan o fallan
- Errores son bugs en código

```
def test_sum():
    assert sum(1, 2) == 3
```

Testing de Datos

- Datos cambiantes
- Input desconocido/ variable
- Tests probabilísticos
- Errores en fuentes

```
SELECT COUNT(*)
WHERE id IS NULL
-- Debería ser 0
```

¿Dónde Testear en el Pipeline?

Sources ⚠️ Test freshness

↓

Staging ⚠️ Test not null, unique

↓

Intermediate ⚠️ Test relationships

↓

Marts ⚠️ Test business rules

¿Dónde Testear en el Pipeline?

Testear en cada capa atrapa errores temprano y facilita debugging

Tests Nativos de dbt

Tipos de Tests en dbt

Generic Tests

Tests pre-construidos que se aplican via YAML:

- `unique` - Sin duplicados
- `not_null` - Sin nulos
- `accepted_values` - Valores permitidos
- `relationships` - Integridad referencial

Fáciles de configurar, cubren casos comunes

Singular Tests

Tests personalizados en SQL:

- Queries que retornan filas con errores
- Lógica de negocio específica
- Validaciones complejas

Más flexibles, requieren escribir SQL

Test: unique

¿Qué valida?

Verifica que no haya valores duplicados en una columna.

Test: unique

Configuración YAML

```yaml
models:
  - name: stg_pokemon
    columns:
      - name: pokemon_id
        tests:
          - unique
```

SQL que dbt ejecuta

```sql
SELECT pokemon_id
FROM stg_pokemon
GROUP BY pokemon_id
HAVING COUNT(*) > 1
-- Falla si retorna filas
```

Test: not_null

¿Qué valida?

Verifica que no haya valores NULL en una columna.

Test: not_null

Configuración YAML

```yaml
models:
  - name: stg_pokemon
    columns:
      - name: pokemon_name
        tests:
          - not_null
```

SQL que dbt ejecuta

```sql
SELECT pokemon_name
FROM stg_pokemon
WHERE pokemon_name IS NULL
-- Falla si retorna filas
```

Test: accepted_values

¿Qué valida?

Verifica que los valores de una columna estén en una lista permitida.

Test: accepted_values

Configuración YAML

```yaml
models:
  - name: obt_pokemon
    columns:
      - name: power_tier
        tests:
          - accepted_values:
              values:
                - 'Legendary'
                - 'Strong'
                - 'Normal'
```

SQL que dbt ejecuta

```sql
SELECT power_tier
FROM obt_pokemon
WHERE power_tier NOT IN (
  'Legendary',
  'Strong',
  'Normal'
)
-- Falla si retorna filas
```

Test: relationships

¿Qué valida?

Verifica integridad referencial - que cada valor en una columna exista en otra tabla.

Test: relationships

Configuración YAML

```yaml
models:
  - name: fct_battles
    columns:
      - name: pokemon_id
        tests:
          - relationships:
              to: ref('dim_pokemon')
              field: pokemon_id
```

Útil para validar foreign keys en modelos dimensionales

Archivo de Tests Completo

models/staging/_stg__models.yml

```yaml
version: 2
models:
  - name: stg_pokemon
    description: "Datos de Pokemon limpiados desde la API"
    columns:
      - name: pokemon_id
        description: "ID único del Pokemon"
        tests:
          - unique
          - not_null
      - name: pokemon_name
        description: "Nombre del Pokemon"
        tests:
          - not_null
      - name: height
        tests:
          - not_null
      - name: weight
        tests:
          - not_null
```

☕ Break

15 minutos

Regresamos para ver dbt-expectations y singular tests

DBT-EXPECTATIONS

¿Qué es dbt-expectations?

Definición

dbt-expectations es un paquete que extiende dbt con 60+ tests adicionales, inspirado en Great Expectations (librería de Python para data quality).

¿Qué es dbt-expectations?

Tests nativos de dbt

- unique
- not_null
- accepted_values
- relationships

Solo 4 tests básicos

Con dbt-expectations

- Rangos numéricos
- Patrones regex
- Distribuciones estadísticas
- Comparación entre columnas
- Y 50+ más...

60+ tests avanzados

Instalación de dbt-expectations

Paso 1: Agregar al packages.yml

Crear archivo `packages.yml` en la raíz del proyecto:

```yaml
packages:
  - package: metaplane/dbt_expectations
    version: 0.10.10
```

Instalación de dbt-expectations

Paso 2: Instalar el paquete

```
dbt deps
```

Esto descarga el paquete a la carpeta `dbt_packages/`

Tests de Rango Numérico

expect_column_values_to_be_between

Verifica que valores estén dentro de un rango.

Tests de Rango Numérico

Ejemplo: altura y peso de Pokemon

```yaml
columns:
  - name: height
    tests:
      - dbt_expectations.expect_column_values_to_be_between:
          min_value: 1
          max_value: 200
  - name: base_experience
    tests:
      - dbt_expectations.expect_column_values_to_be_between:
          min_value: 0
          max_value: 1000
```

Tests de Formato y Patrón

expect_column_values_to_match_regex

Verifica que valores cumplan con un patrón regex.

Ejemplo: nombres solo letras y guiones

```yaml
columns:
  - name: pokemon_name
    tests:
      - dbt_expectations.expect_column_values_to_match_regex:
          regex: "^[a-z\\-]+$"
```

expect_column_values_to_be_in_type_list

```yaml
- name: pokemon_id
  tests:
    - dbt_expectations.expect_column_values_to_be_in_type_list:
        column_type_list: [INTEGER, BIGINT]
```

Tests de Completitud

expect_column_to_exist

Verifica que una columna exista en la tabla.

expect_table_row_count_to_be_between

```yaml
models:
  - name: stg_pokemon
    tests:
      - dbt_expectations.expect_table_row_count_to_be_between:
          min_value: 100
          max_value: 2000
```

expect_column_proportion_of_unique_values_to_be_between

```yaml
- name: pokemon_name
  tests:
    - dbt_expectations.expect_column_proportion_of_unique_values_to_be_between:
        min_value: 0.95  # 95% valores únicos
```

expect_column_mean_to_be_between

```yaml
- name: base_experience
  tests:
    - dbt_expectations.expect_column_mean_to_be_between:
        min_value: 50
        max_value: 200
```

expect_column_stdev_to_be_between

```yaml
- name: height
  tests:
    - dbt_expectations.expect_column_stdev_to_be_between:
        min_value: 1
        max_value: 50
```

Útil para detectar anomalías en distribuciones de datos

Tests Más Usados de dbt-expectations

Test	Uso
expect_column_values_to_be_between	Rangos numéricos
expect_column_values_to_match_regex	Patrones de texto
expect_table_row_count_to_be_between	Tamaño de tabla
expect_column_values_to_be_in_set	Lista de valores permitidos
expect_column_pair_values_A_to_be_greater_than_B	Comparar columnas
expect_column_values_to_not_be_null	No nulos (alias)
expect_compound_columns_to_be_unique	Unicidad compuesta
expect_column_distinct_count_to_be_between	Cardinalidad

Singular Tests

¿Qué son los Singular Tests?

Definición

Singular tests son queries SQL personalizados que validan reglas de negocio específicas. Se guardan en la carpeta `tests/`.

Regla: El test **falla** si el query retorna **alguna fila**.

¿Qué son los Singular Tests?

Estructura

```
mi_proyecto_dbt/
└── tests/
    ├── assert_pokemon_positive_stats.sql
    └── assert_no_duplicate_battles.sql
```

Ejemplo: Validar Stats Positivos

tests/assert_pokemon_positive_stats.sql

```sql
-- Test falla si encuentra Pokemon con stats negativos
SELECT
    pokemon_id,
    pokemon_name,
    height,
    weight,
    base_experience
FROM {{ ref('stg_pokemon') }}
WHERE height < 0
   OR weight < 0
   OR base_experience < 0
```

Ejemplo: Validar Stats Positivos

Si el query retorna 0 filas → Test PASA
Si retorna 1+ filas → Test FALLA

Ejemplo: Validar Regla de Negocio

tests/assert_legendary_high_experience.sql

```sql
-- Regla: Pokemon "Legendary" deben tener base_experience >= 200

SELECT
    pokemon_id,
    pokemon_name,
    power_tier,
    base_experience
FROM {{ ref('obt_pokemon') }}
WHERE power_tier = 'Legendary'
  AND base_experience < 200
```

Ejemplo: Validar Regla de Negocio

Los singular tests son ideales para validar **lógica de negocio** que no se puede expresar con tests genéricos

Ejemplo: Validar Freshness Manual

tests/assert_data_freshness.sql

```sql
-- Verificar que los datos no tengan más de 24 horas

SELECT
    MAX(_airbyte_extracted_at) as last_load,
    CURRENT_TIMESTAMP as now,
    DATEDIFF('hour', MAX(_airbyte_extracted_at),
CURRENT_TIMESTAMP) as hours_old
FROM {{ ref('stg_pokemon') }}
HAVING DATEDIFF('hour', MAX(_airbyte_extracted_at),
CURRENT_TIMESTAMP) > 24
```

Ejemplo: Validar Freshness Manual

Útil cuando no podés usar source freshness de dbt

Ejecutar Tests

Comandos de Testing

Comando	Descripción
`dbt test`	Ejecuta todos los tests
`dbt test --select stg_pokemon`	Tests de un modelo específico
`dbt test --select staging.*`	Tests de todos los modelos en staging
`dbt test --select test_type:singular`	Solo singular tests
`dbt test --select test_type:generic`	Solo generic tests
`dbt build`	Run + test en orden de dependencias

Recomendación: Usar `dbt build` en lugar de `dbt run` + `dbt test` por separado

Output típico de dbt test

```
21:30:00  Running 8 tests
21:30:01  1 of 8 PASS unique_stg_pokemon_pokemon_id ............. [PASS in 0.15s]
21:30:01  2 of 8 PASS not_null_stg_pokemon_pokemon_id ........... [PASS in 0.12s]
21:30:01  3 of 8 PASS not_null_stg_pokemon_pokemon_name ......... [PASS in 0.11s]
21:30:02  4 of 8 FAIL accepted_values_obt_pokemon_power_tier ..... [FAIL 3 in 0.18s]
21:30:02  5 of 8 PASS expect_column_values_to_be_between ......... [PASS in 0.14s]

Completed with 1 error:

Failure in test accepted_values_obt_pokemon_power_tier
  Got 3 results, configured to fail if != 0
```

Configurar Severidad de Tests

Niveles de severidad

- `error` (default): Falla el pipeline
- `warn`: Solo muestra advertencia, no falla

Configurar Severidad de Tests

Configurar en YAML

```yaml
columns:
  - name: pokemon_name
    tests:
      - not_null:
          severity: error  # Crítico - debe fallar
      - unique:
          severity: warn   # Advertencia - no falla el build
```

Usar `warn` para tests informativos que no bloquean el pipeline

Documentación en dbt

¿Por Qué Documentar?

Beneficios de la documentación

- **Autodescubrimiento:** Usuarios encuentran datos sin preguntar
- **Confianza:** Saben qué significan los campos
- **Mantenimiento:** Facilita onboarding de nuevos miembros
- **Gobernanza:** Trazabilidad y linaje de datos

En dbt, la documentación vive **junto al código** - no en wikis separadas que se desactualizan

Documentar Modelos y Columnas

En el archivo YAML

```yaml
models:
  - name: obt_pokemon
    description: >
      One Big Table con todos los datos de Pokemon para análisis.
      Incluye información básica, tipos, habilidades y clasificación de poder.
    columns:
      - name: pokemon_id
        description: "Identificador único del Pokemon en la PokeAPI"
      - name: power_tier
        description: >
          Clasificación de poder basada en base_experience:
          - Legendary: >= 200
          - Strong: >= 100
          - Normal: < 100
```

Generar y Servir Documentación

Comandos

```
# Generar documentación estática
dbt docs generate

# Servir localmente (abre navegador)
dbt docs serve

# Servir en puerto específico
dbt docs serve --port 8001
```

Generar y Servir Documentación

Lo que incluye

- Descripciones de modelos y columnas
- Tests configurados
- **DAG interactivo** de dependencias
- Código SQL compilado
- Linaje de datos

El DAG Interactivo

Visualización de dependencias

El DAG (Directed Acyclic Graph) muestra:

- **Sources** (verde): Tablas externas
- **Models** (azul): Transformaciones dbt
- **Tests** (naranja): Validaciones
- **Flechas**: Dependencias (ref, source)

El DAG Interactivo

Podés hacer click en cualquier nodo para ver:

- Descripción
- Código SQL
- Columnas y tests
- Dependencias upstream/downstream

Actividad Asincrónica

Tarea Práctica - Clase 6

Objetivos

Agregar testing y documentación completa a tu proyecto dbt.

Tarea Práctica - Clase 6

Entregable

Actualizar tu repositorio Git con:

1. **dbt-expectations** instalado (`packages.yml` + `dbt deps`)
2. Al menos **5 tests genéricos** (unique, not_null, etc.)
3. Al menos **3 tests de dbt-expectations**
4. Al menos **2 singular tests** personalizados
5. **Documentación** de todos los modelos y columnas clave
6. Captura del **DAG** con documentación generada

Todos los tests deben pasar con `dbt build`

Tests Mínimos Requeridos

Capa	Test requerido	Tipo
staging	unique + not_null en PKs	Generic
staging	expect_table_row_count_to_be_between	dbt-expectations
intermediate	relationships (si aplica)	Generic
marts	accepted_values en categorías	Generic
marts	expect_column_values_to_be_between	dbt-expectations
tests/	Regla de negocio personalizada	Singular

Foro de Discusión - Clase 6

Consigna: Problemas de calidad de datos

Comparte tu experiencia con **problemas de calidad de datos**:

1. ¿Qué **errores** encontraste al trabajar con datos?
2. ¿Cómo **investigaste** la causa raíz del problema?
3. ¿Cómo lo **solucionaste**?
4. ¿Qué **tests** te hubieran servido en esa ocasión?

Si no encontraste errores, comparte qué tests consideras **más críticos** para datos de producción.

Comenta al menos una publicación de un compañero.

Foro de Discusión - Clase 6

⏰ *Participar antes de la próxima clase*

¿Preguntas?

📧 rmparra@pol.una.py

📚 Material en plataforma

💬 Foro de consultas abierto

Resumen Clase 6

Conceptos clave

- Dimensiones de calidad de datos
- Generic tests: unique, not_null, accepted_values, relationships
- dbt-expectations: 60+ tests
- Singular tests: SQL personalizado
- Severidad: error vs warn

Práctica

- Configurar tests en YAML
- Instalar dbt-expectations
- Escribir singular tests
- Ejecutar dbt test / dbt build
- Generar documentación
- Explorar el DAG

Resumen Clase 6

Próxima clase: Orquestación y Visualización

Prefect para scheduling + Metabase para dashboards